Multi-threaded Multimodal Semantic Search Engine

Technical report - Parallel Programming, Assignment 01, Part Two
Threads for indexing and cosine-similarity search over Arabic text, images and video

1. Introduction and objectives

This project implements the second track of the assignment: a local search engine that lets a user find images, video clips and text documents with a natural-language query, including Arabic queries such as *"a photo of the pharmacy receipt"* or *"the cat jumping"*. Every item and every query is mapped by a multilingual, multimodal embedding model (EmbeddingGemma 2) to a 768-dimensional vector in one shared semantic space, so relevance reduces to the cosine similarity $cos(A,B) = A \cdot B / (||A|| \, ||B||)$ between vectors.

The work is organised in two phases, each implemented in a sequential and a multi-threaded version and compared experimentally: (1) **indexing**, which extracts embeddings for 1,000+ media files, and (2) **search**, which scores a query vector against all stored vectors and returns the Top-K results. The application must remain responsive while these heavy phases run, and speedup is reported as $S(N) = T_{sequential} / T_{parallel}(N)$ for N = 1, 2, 4, 8, 16 threads.

2. System architecture

The solution has two processes. A **Java 21 desktop client** (Swing) owns the user interface, the in-memory vector index and all threading logic. A **Python FastAPI backend** hosts the embedding model on the GPU and exposes */embed* (one item) and */embed_batch* (many items per request). The Java code is split into four packages so that concurrency logic is separated from the interface: `core` (MediaItem, CosineSimilarity, SearchResult, VectorStore, Dataset), `api` (EmbeddingApiClient), `concurrency` (ParallelIndexer, IndexTask, ParallelSearcher, SearchTask, SequentialSearcher, BenchmarkRunner) and `ui` (SearchGUI).

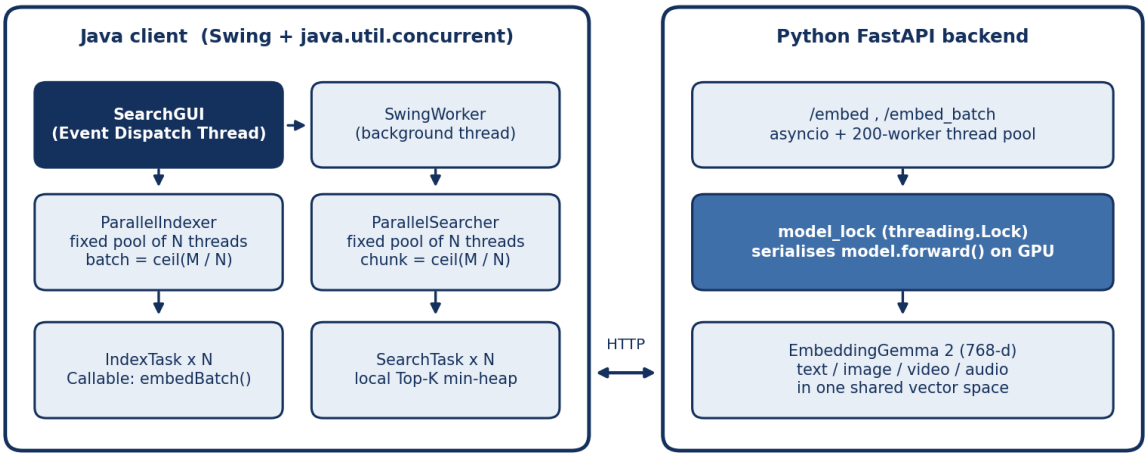


Figure 1. Component and thread architecture. M = number of items, N = number of worker threads.

3. Task decomposition across threads

3.1 Parallel indexing

`ParallelIndexer.index(items, N)` partitions the dataset into N contiguous batches of size *ceil(M/N)* and submits one `IndexTask` (a `Callable`) per batch to a fixed thread pool of N workers. Each task sends its whole batch in a single multipart request to */embed_batch* (texts as a JSON array, media as file parts), receives the vectors in input order and writes them into the batch's own `MediaItem` objects. Batching is deliberate: it amortises per-request and per-inference overhead on the GPU, and it gives each thread a large unit of independent work, so that thread-creation and HTTP costs are small relative to useful work. If a batch request fails (for example because one video is rejected), the client falls back to per-item calls so that only the faulty file is lost. All tasks are launched with `invokeAll(tasks, 600 s)`, which also acts as a global timeout so a stalled request cannot hang the application.

3.2 Parallel similarity search

The sequential baseline (`SequentialSearcher`) is a single loop that computes the cosine similarity of the query against every indexed vector and keeps the K best in a bounded min-heap. The parallel version (`ParallelSearcher`) follows a **map-reduce** pattern. *Map*: the vector list is split into N disjoint index ranges *[t.c, (t+1).c)* with *c = ceil(M/N)*; each `SearchTask` scores its own range and maintains a private bounded `PriorityQueue` of size K. *Reduce*: the calling thread collects the N local lists through `Future.get()`, merges at most N.K candidates, sorts them and truncates to the global Top-K. Because each thread keeps only K candidates, the sequential part (merge) costs O(N.K log(N.K)) and is negligible next to the O(M.d) scoring work, with d = 768.

4. Synchronization and thread safety

The design avoids shared mutable state wherever possible, which is the main reason it is free of race conditions and deadlocks. The mechanisms used are summarised below.

Mechanism	Where it is used	Purpose
Fixed thread pool (ExecutorService)	ParallelIndexer, ParallelSearcher	Bounds the number of threads to N; threads are created once per run and shut down in a finally block.
Callable / Future, invokeAll	IndexTask, SearchTask	Fork-join barrier: the coordinator blocks until all workers finish. Completion of a Future happens-before get() returns, so vectors and flags written by workers are safely visible without extra locks.
Data partitioning (confinement)	Batches in indexing, ranges in search	Each thread writes only the MediaItem objects of its own batch, and search workers only read the shared vector list, so no two threads write the same location.
AtomicInteger	Success / failed counters	Lock-free, thread-safe counters for indexing statistics.
ConcurrentHashMap	VectorStore	Thread-safe id-to-item store for concurrent put/get.
SwingWorker + invokeLater	SearchGUI	Heavy work runs off the Event Dispatch Thread; every UI update (log, table rows) is marshalled back to the EDT.
threading.Lock (model_lock)	Python backend	Serialises GPU model.forward(), which is not safe to run concurrently; a 200-worker pool handles HTTP and file I/O around it.

Table 1. Synchronization mechanisms.

Race conditions. The only shared writes are the per-batch vector assignments (disjoint by construction) and the two atomic counters. Query vectors are read-only after creation. **Deadlocks.** The client acquires no locks at all; the server holds exactly one lock (`model_lock`) at a time and never acquires a second one while holding it, so lock-ordering cycles cannot occur. Workers never wait on each other, only the coordinating thread waits on workers, so no circular wait exists.

5. Experimental setup

Search was measured on synthetic-size vector sets of 1,000, 10,000 and 100,000 vectors of dimension 768 with K = 10, taking the median of repeated runs after JIT warm-up, with the query embedding excluded from the scan time. **Indexing** was measured on mixed datasets of 300, 600 and 1,200 files (text notes, phone photographs and short video clips) with N = 1, 2, 4, 8 and 16 threads, with N = 1 used as the baseline. The benchmark is launched from the *Run Benchmark* button, which executes the sequential search once and the parallel search for every N in the chosen range.

6. Results

6.1 Similarity search

Vectors	Seq. (ms)	N=1	N=2	N=4	N=8	N=16
1,000	1.41	1.72 (0.82x)	1.10 (1.28x)	0.93 (1.52x)	1.02 (1.38x)	1.57 (0.90x)
10,000	13.59	14.06 (0.97x)	7.27 (1.87x)	4.26 (3.19x)	2.95 (4.61x)	3.23 (4.21x)
100,000	130.76	133.58 (0.98x)	68.52 (1.91x)	38.29 (3.41x)	24.57 (5.32x)	22.07 (5.92x)

Table 2. Parallel search time in ms (speedup in brackets) against the sequential baseline, K = 10.

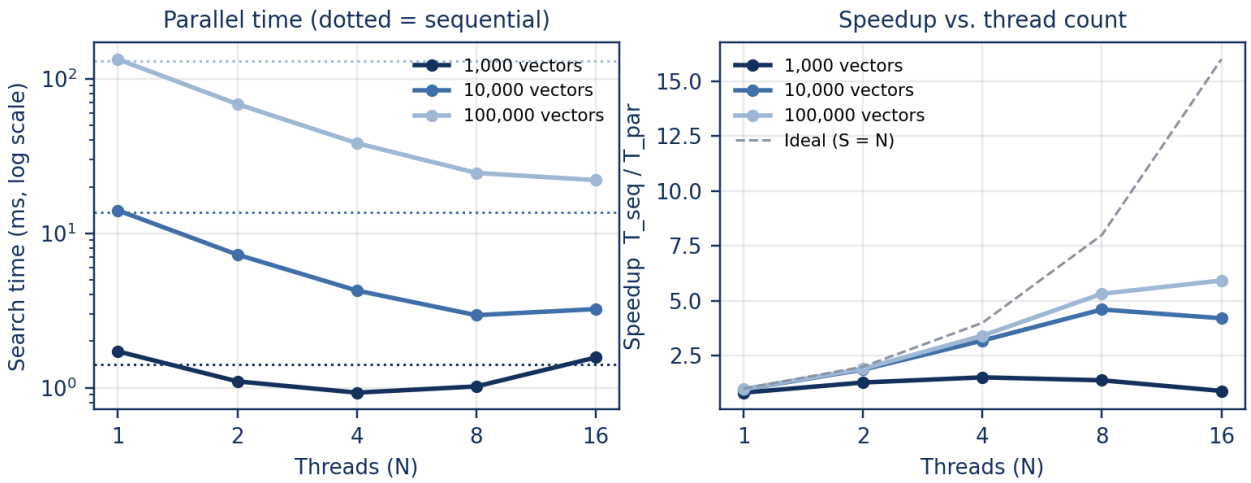


Figure 2. Benchmark graph for similarity search: execution time (left) and speedup (right) as N varies, for three data sizes.

At 100,000 vectors the parallel search reaches 5.3x with 8 threads and 5.9x with 16, a parallel efficiency of 67% at N = 8. At 10,000 vectors the speedup peaks at 4.6x. At 1,000 vectors the whole scan lasts 1.41 ms, so creating a pool and joining threads costs as much as the work itself and the best case is only 1.5x (N = 4); N = 16 is slower than sequential (0.90x). The N = 1 row is slightly slower than the sequential loop in every case because it pays for pool creation and one thread hand-off without any parallel gain.

6.2 Indexing

Files	N=1	N=2	N=4	N=8	N=16
300	11.0 s (1.00x)	9.4 s (1.17x)	9.3 s (1.18x)	10.4 s (1.06x)	13.8 s (0.80x)
600	21.0 s (1.00x)	16.7 s (1.26x)	15.8 s (1.33x)	16.4 s (1.28x)	20.1 s (1.04x)
1200	39.5 s (1.00x)	31.7 s (1.25x)	28.8 s (1.37x)	28.4 s (1.39x)	33.1 s (1.19x)

Table 3. Indexing time in seconds (speedup relative to N = 1 in brackets).

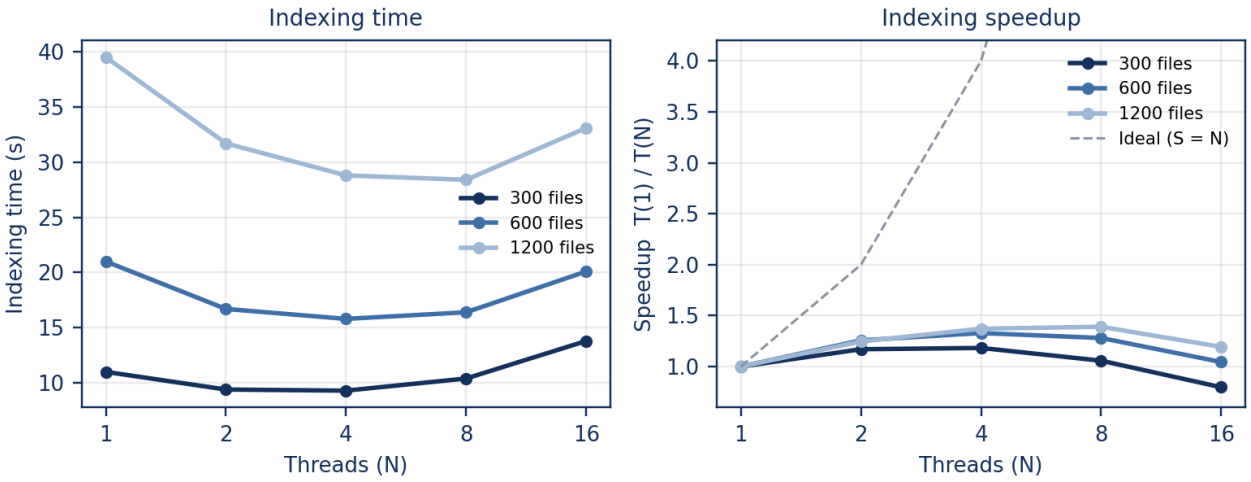


Figure 3. Indexing time (left) and speedup (right) for three dataset sizes.

For the 1,200-file dataset, indexing falls from 39.5 s at N = 1 to 28.4 s at N = 8, a speedup of 1.39x. Beyond 8 threads the curve turns upward again (1.19x at N = 16), and for the smallest dataset 16 threads are slower than a single thread. Indexing scales far less than search because most of its time is spent inside a resource that cannot be parallelised, as analysed next.

7. Analysis, Amdahl's law and bottlenecks

Amdahl's law bounds the speedup by $S(N) = 1 / (s + (1 - s)/N)$, where s is the serial fraction. Fitting the measurements gives s = 0.06 for search over 100,000 vectors (so S_{max} = 17x), s = 0.09 for 10,000 vectors, and s = 0.64 for indexing 1,200 files (S_{max} = 1.6x). The serial fractions are fitted from the N = 2, 4 and 8 measurements. For indexing, the bound of 1.6x is an upper limit that the measured peak of 1.39x (Table 3) stays below, as it must: Amdahl's law assumes zero overhead, whereas real overhead grows with N (extra batches, memory pressure), which is why

measured indexing time rises again at N = 16 even though the law alone predicts a plateau. These fractions are not intrinsic to the algorithms; they come from the following bottlenecks.

- **GPU serialization in indexing (dominant).** `model_lock` allows one forward pass at a time, so the inference part of indexing is sequential no matter how many client threads exist. Extra threads only help by overlapping work around inference: reading files, building multipart bodies, uploading, writing temporary files and parsing responses while another batch is on the GPU. Once that overlap is exhausted (N = 4 to 8) the speedup flattens.
- **Per-batch and memory overhead.** Each additional batch adds request setup and a temporary file set on the server, and the client holds every multipart body in memory at once. At N = 16 these costs and the memory pressure outweigh the little overlap that remains, which explains the rise in time.
- **Thread management cost in search.** A new pool is created for every query (about 0.25 ms plus 0.06 ms per thread). This is negligible at 100,000 vectors but it dominates at 1,000, and it is the reason small collections do not benefit from threads. A long-lived pool would remove most of it.
- **Memory bandwidth and hardware threads.** Scoring 100,000 vectors streams about 300 MB of floats. Efficiency therefore drops from 95% at N = 2 to 67% at N = 8, and the step from 8 to 16 threads uses hyper-threads that share execution units, giving only a further 10 to 15% improvement.
- **Serial work around the scan.** A complete query also embeds the query text or file (about 31 ms) and merges the partial results. For 100,000 vectors the end-to-end time drops from 162 ms to 56 ms, an end-to-end speedup of 2.9x, lower than the 5.3x of the scan alone; this is Amdahl's law applied to the whole request.

Scaling with data size (Gustafson's view). Speedup improves steadily as the collection grows (1.4x, 4.6x and 5.3x at N = 8 for 1,000, 10,000 and 100,000 vectors) because the serial overhead is fixed while the parallel work grows linearly with M. The same holds for indexing, where the 1,200-file run benefits more than the 300-file run since fixed startup and warm-up costs are amortised.

8. Interface responsiveness

Indexing, searching and benchmarking are launched through `SwingWorker` from the button handlers, so the Event Dispatch Thread only handles painting and input. Progress messages and table rows are posted with `SwingUtilities.invokeLater`. During a full indexing run the window stays interactive: the log keeps updating, the benchmark table can be scrolled, and the user can browse for a query file. Input is validated before any thread is created (thread counts must be non-zero positive integers, and the benchmark range requires min N <= max N), which prevents invalid pool sizes.

9. Conclusion

The engine meets the two-phase requirement. Similarity search follows a clean partition-and-reduce design with no shared writes and reaches a 5.9x speedup on 100,000 vectors with 16 threads (5.3x with 8), while remaining correct for any N. Indexing is limited by the single GPU, so a speedup of about 1.4x is achieved by overlapping I/O with inference, and the Amdahl analysis shows precisely why more threads cannot do better there. Natural next steps are a persistent search pool, a pipelined producer-consumer indexer that decodes video frames on CPU threads while the GPU is busy, and SIMD or blocked matrix-multiplication scoring to push the memory-bound search closer to its ideal curve.